

CODING ASSESSMENT

Large-Scale Image Processing — Console EXE

.NET / C# Backend Engineer

Objective: Build a production-minded console application that processes a large collection of images efficiently, persists results in SQLite, and exposes a CLI interface — demonstrating sound architecture, concurrency, memory management, and engineering judgment.

1. Problem Statement

Design and implement a .NET console EXE for processing a directory of potentially very large image files. The system should discover images, extract metadata, generate thumbnails, calculate SHA-256 hashes, detect duplicates, persist results in SQLite, and provide a CLI to monitor and query results. The key challenge is remaining controlled as the number and size of files grows — avoid loading the entire dataset into memory or creating unbounded concurrency.

2. Input & Image Processing

Use `./data/images/` as input. The directory may contain thousands of images of varying sizes. Direct files only; recursive scanning not required.

- Discover supported image files without loading the entire directory into memory.
- Extract at minimum: file name, path, file size, extension/format, width and height.
- Calculate a SHA-256 hash for duplicate detection.
- Handle large files without retaining their complete byte contents in memory.
- Choose a suitable image-processing library and document your choice.

3. Thumbnail Generation

Generate a thumbnail (max **300 × 300 px**, aspect ratio preserved) for each successfully processed image. Store under `./data/thumbnails/`. Format and naming are your choice; must be deterministic and safe for concurrent processing.

4. Processing Pipeline

File Discovery → Work Queue → N Concurrent Workers → Result Handling → SQLite Persistence

Choose your concurrency mechanism (Channels, TPL, producer/consumer, etc.) and explain the choice.

- Use bounded queues — a huge input directory must not cause unbounded in-memory growth.
- Avoid one long-running thread/task per image.
- Use async I/O where it provides a clear benefit.
- Make concurrency configurable (worker count, queue capacity) rather than hard-coded.

5. SQLite Persistence

Persist results in SQLite with at minimum: *Id, FileName, FilePath, FileSize, Width, Height, Format, SHA256, ThumbnailPath, ProcessingStatus, ErrorMessage, CreatedAt, ProcessedAt*.

- Use parameterized queries.
- Batch writes and use transactions where appropriate.
- Add sensible indexes for queries the CLI exposes.
- Consider concurrent readers/writers and SQLite write limitations.

6. Duplicate Detection

Use SHA-256 to identify duplicates. The CLI should allow users to determine whether an image has duplicates or belongs to a duplicate group.

7. CLI Interface

Expose functionality via command-line arguments/subcommands. Required commands:

Command	Description
<code>process [--workers N]</code>	Start image processing; print progress to stdout
<code>status</code>	Show current/last job status and progress
<code>list [--page N] [--size N]</code>	Paginated list of processed images
<code>search --name X / --format X</code>	Filter processed images
<code>info --id X</code>	Retrieve details for one image
<code>thumbnail --id X</code>	Print thumbnail path for an image

cancel	Cancel an active processing run
--------	---------------------------------

8. Failure Handling

- A corrupt or unsupported image must not terminate the entire job.
- Record a failed status and useful error info per image.
- Handle missing/deleted files gracefully.
- Avoid partially written thumbnail/database records where possible.

9. Cancellation

Implement graceful cancellation using .NET cancellation tokens. Cancellation should stop new work and allow in-progress operations to finish or terminate safely.

10. Optional Resume

Support resuming an interrupted job after restart (optional — don't sacrifice core correctness).

12. Logging

Use `ILogger<T>` or equivalent. Log job start/end, progress, failures, cancellation, and persistence batches. Avoid per-byte logging.

11. Configuration

Externalize worker count, queue capacity, SQLite batch size, thumbnail dimensions. Keep env-specific values out of business logic.

13. Tests

- Metadata extraction
- Duplicate detection
- Invalid/corrupt image handling
- Pagination/search behavior
- At least one concurrency behavior test

14. Performance & Scale

Must comfortably handle **~5,000 mixed-size images**, including files larger than 20 MB. Do not read all files into memory, queue unbounded full image objects/byte arrays, or create uncontrolled concurrency. In your README, explain what you would change or monitor if the input grew to **1,000,000 images**.

15. Deliverables

- Source-code repository
- README with setup/run instructions
- Automated tests
- Sample dataset or generation instructions
- Architecture note: concurrency, memory, SQLite, failure, cancellation, trade-offs

16. Scope Guidance

Core correctness + controlled concurrency + bounded memory + reliable SQLite are more important than feature count.

Nice-to-have: resume, retry policies, progress metrics, benchmark. Skip: auth, Docker/K8s, CI/CD, cloud, microservices, message brokers.

17. What We Will Look For

- C#.NET fundamentals and clean code.
- Understanding of async/concurrency and backpressure.
- SQLite/data-access decisions.
- Meaningful tests.
- Practical architecture and separation of concerns.
- Memory-conscious handling of large files and collections.
- Robust error handling and cancellation.
- Engineering judgment and useful creativity.

Good luck — we are interested not only in whether the application works, but in how you think about building it.